

검색어 자동완성 시스템 설계

구글 검색창의 autocomplete/typeahead 기능 구현

숭실대학교 소프트웨어학부 21학번 김현수(<https://linktr.ee/rover0811>)

이 발표로부터 무엇을 얻어갈 수 있는가?

- 쿠버네티스에 대한 짚막한 이해
- 시스템 설계에 대한 이해
- 트라이 자료구조에 대한 이해
- CQRS에 대한 이해

문제 정의

목표: 많이 이용된 검색어 k개를 자동완성하여 출력하는 시스템 설계

핵심 질문들

- 자동완성 검색어 개수: 5개
- 선정 기준: 질의 빈도에 따른 검색어 인기순위
- 지원 언어: 영어 (다국어 확장 고려)
- 문자 처리: 소문자만 지원
- 사용자 규모: DAU(Daily Active Users) 1000만명

요구사항

기능적 요구사항

- 빠른 응답속도: 100ms 이내
- 연관성: 입력 단어와 관련된 검색어 제안
- 정렬: 인기도 기반 순위 모델

비기능적 요구사항

- 규모 확장성: 높은 트래픽 처리
- 고가용성: 장애 상황에서도 서비스 지속

개략적 규모 추정

기본 가정

- DAU: 1000만명
- 사용자당 일일 검색: 10건
- 검색어당 평균 글자수: 20바이트
- 검색 입력시마다 API 호출 발생

계산 결과

- QPS: 24,000 (최대 48,000)
- 신규 데이터: 일일 0.4GB
- 전체 로그: 일일 1.5TB

초기 설계안: 두 부분 구성

1. 데이터 수집 서비스

- 사용자 입력 질의를 실시간 수집
- 빈도 테이블 구성 (key: 검색어, value: 빈도)

2. 질의 서비스

- 주어진 질의에 대해 상위 5개 인기 검색어 반환
- SQL 기반 조회

```
SELECT * FROM frequency_table  
WHERE query LIKE 'prefix%'  
ORDER BY frequency DESC LIMIT 5
```

질의	빈도	질의: twitch		질의: twitter		질의: twitter		질의: twillo	
		질의	빈도	질의	빈도	질의	빈도	질의	빈도
		twitch	1	twitch	1	twitch	1	twitch	1
				twitter	1	twitter	2	twitter	2
								twillo	1

그림 13-2

초기 설계의 문제점

성능 이슈

- 정규식 검색: $O(n)$
- 정렬 연산: $O(n \log n)$
- 전체 복잡도: $O(n \log n)$

결론

- QPS 24,000 지원에는 부적합
- 매번 정렬 수행은 비효율적
- 더 나은 자료구조 필요

해결책: 트라이(Trie) 자료구조

트라이의 특징

- 문자열 저장에 특화된 트리 구조
- 루트 노드: 빈 문자열
- 각 노드: 글자 하나 + 자식 노드들
- 검색 시간 복잡도: $O(p)$ (p = 접두어 길이)

핵심 아이디어

- 각 노드에 빈도수 저장
- 접두어 기반 빠른 검색

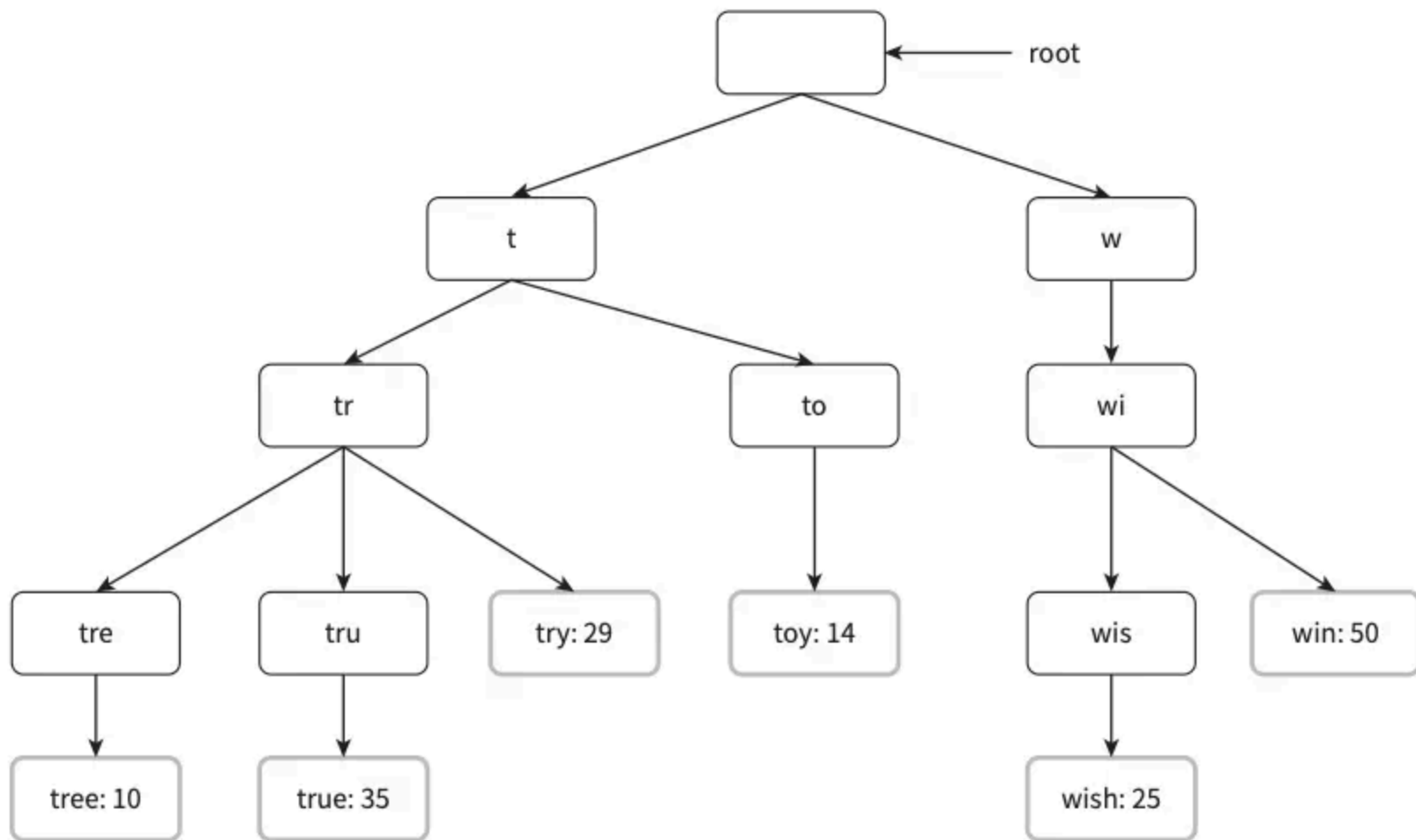


그림 13-6

트라이 최적화 방안

문제: 하위 트리 전체 탐색 + 정렬 필요

해결책 1: 접두어 길이 제한

- 최대 길이 50자로 제한
- 시간 복잡도: $O(1)$

해결책 2: 노드별 인기 검색어 캐시

- 각 노드에 상위 k개 검색어 미리 저장
- 공간 복잡도 증가, 시간 복잡도 $O(1)$

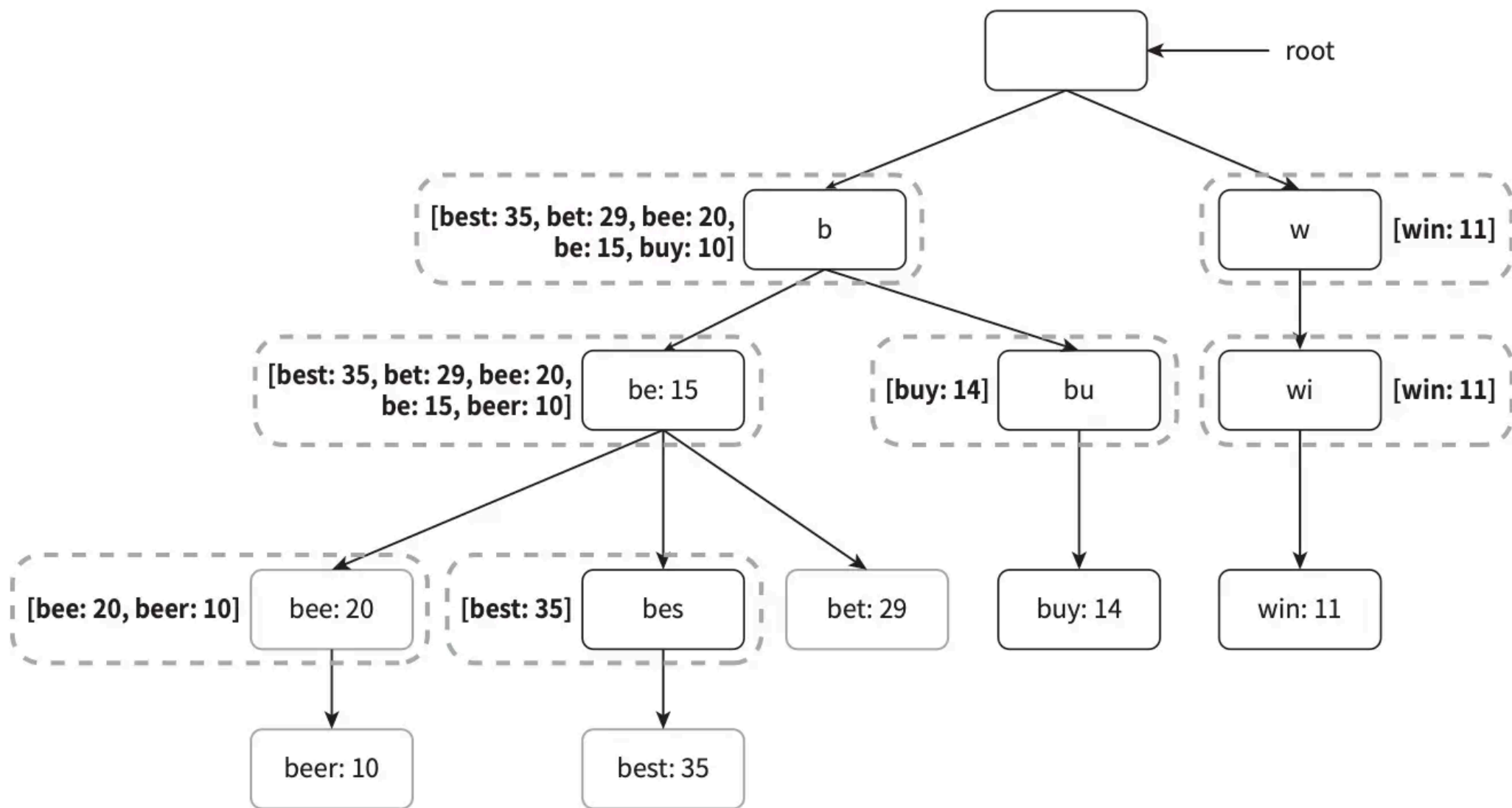
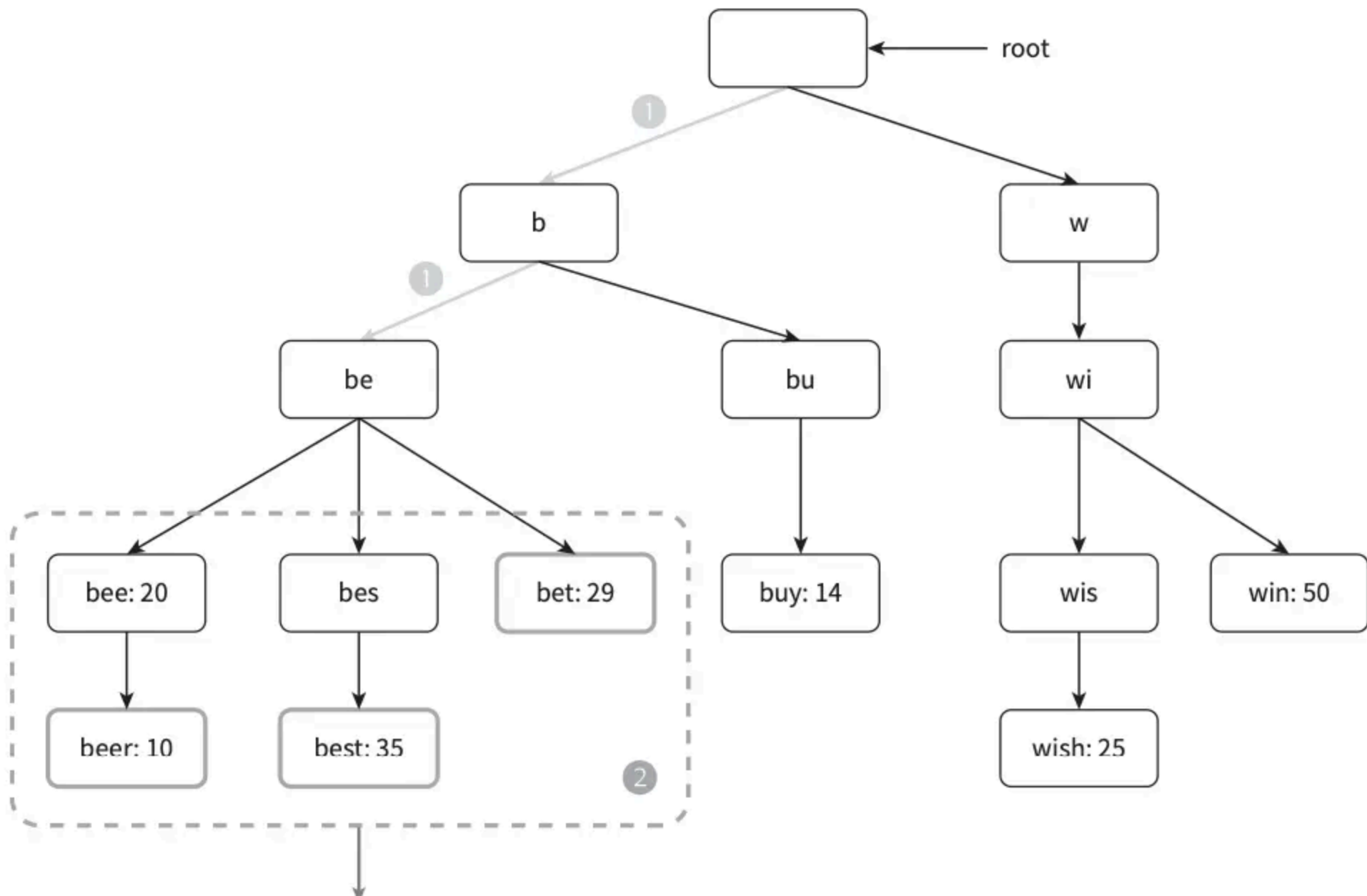


그림 13-8

트라이 검색 과정

예시: "be" 입력시 상위 2개 검색어 찾기

1. 접두어 노드 "be" 탐색
2. 하위 트리의 유효 노드 확인
 - beer: 10, best: 35, bet: 29
3. 정렬하여 상위 k개 반환
 - best: 35, bet: 29



상세 설계: 전체 아키텍처

주요 컴포넌트

1. 데이터 분석 서비스 로그
2. 로그 취합 서버
3. 작업 서버
4. 트라이 데이터베이스 (쓰기용)
5. 트라이 캐시 (읽기용)
6. 질의 서비스

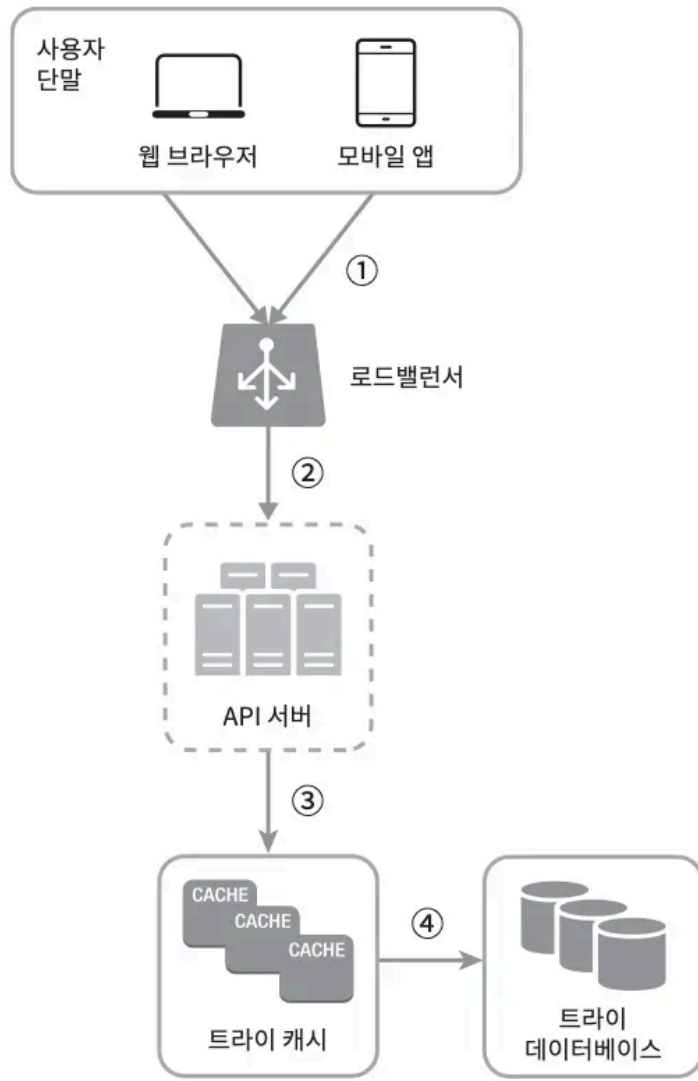


그림 13-11

데이터 흐름

데이터 수집

- 로그 서비스: 일일 1.5TB 데이터 수집
- 로그 취합: 검색어별 빈도 집계
- 작업 서버: 주기적 트라이 재구성

데이터 서빙

- 트라이 캐시: 메모리 기반 고속 조회
- 질의 서비스: 100ms 이내 응답

query	time
tree	2019-10-01 22:01:01
try	2019-10-01 22:01:05
tree	2019-10-01 22:01:30
toy	2019-10-01 22:02:22
tree	2019-10-02 22:02:42
try	2019-10-03 22:03:03

II 13-3

트라이 저장 방식

1. 문서 저장소 (MongoDB)

- 트라이 직렬화하여 저장
- 주기적 스냅샷 생성

2. 키-값 저장소 (Redis)

- 접두어를 키로 변환
- 노드 데이터를 값으로 저장

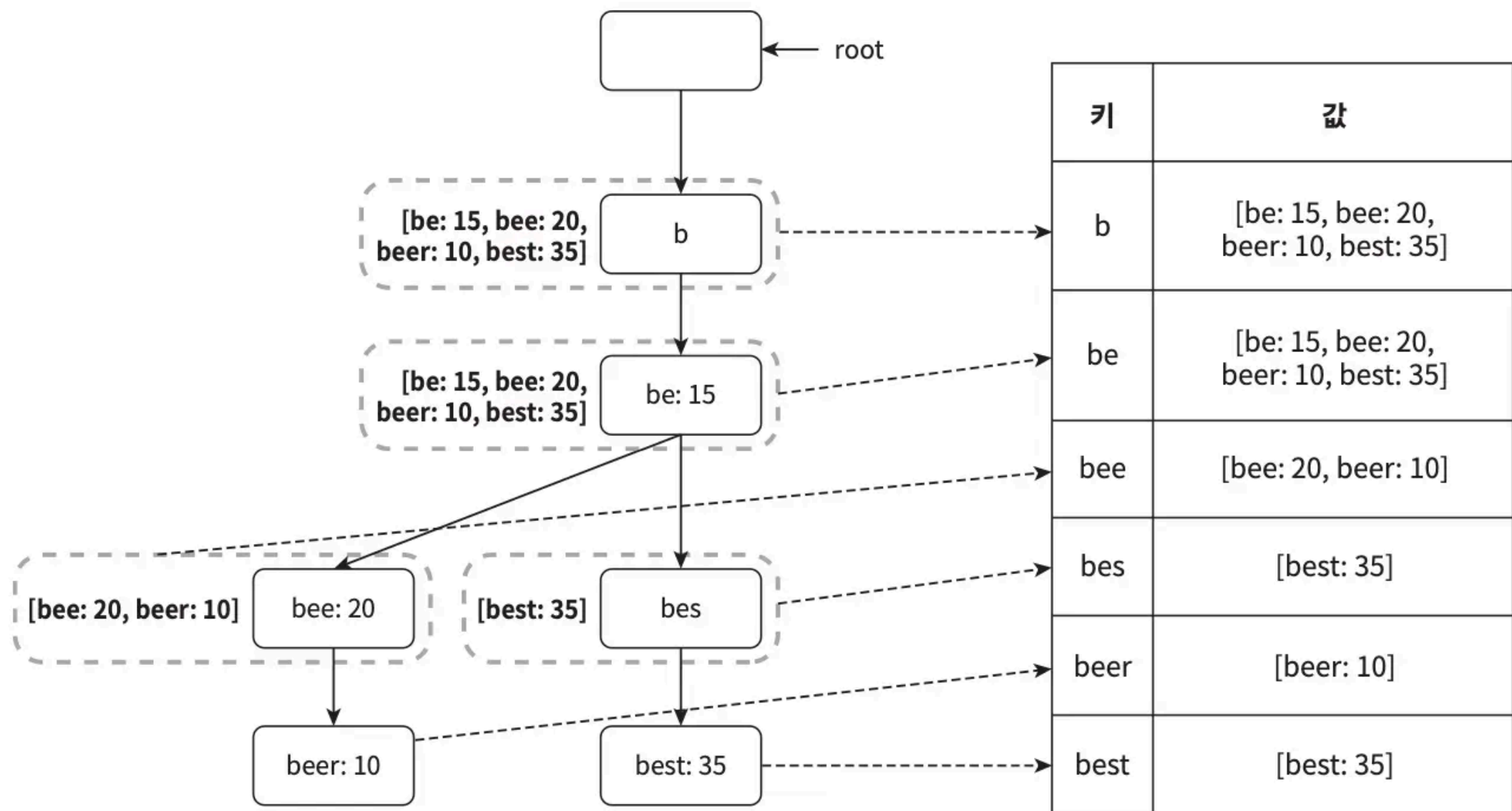


그림 13-10

질의 서비스 최적화

성능 향상 기법

1. AJAX 요청: 페이지 새로고침 없이 데이터 갱신
2. 브라우저 캐싱: 자주 변하지 않는 결과 캐시
3. 데이터 샘플링: N개 요청 중 1개만 로깅

응답 속도

- 목표: 100ms 이내
- 캐시 히트시: $O(1)$ 응답

트라이 갱신 전략

매주 갱신 방식

- 기존 + 신규 데이터 병합
- 새로운 트라이 생성
- 읽기/쓰기 트라이 분리

장점

- 안정적인 성능
- 확장성 우수

단점

- 실시간 반영 제한

로컬 8 CPU 환경 부하 테스트

시스템 구성

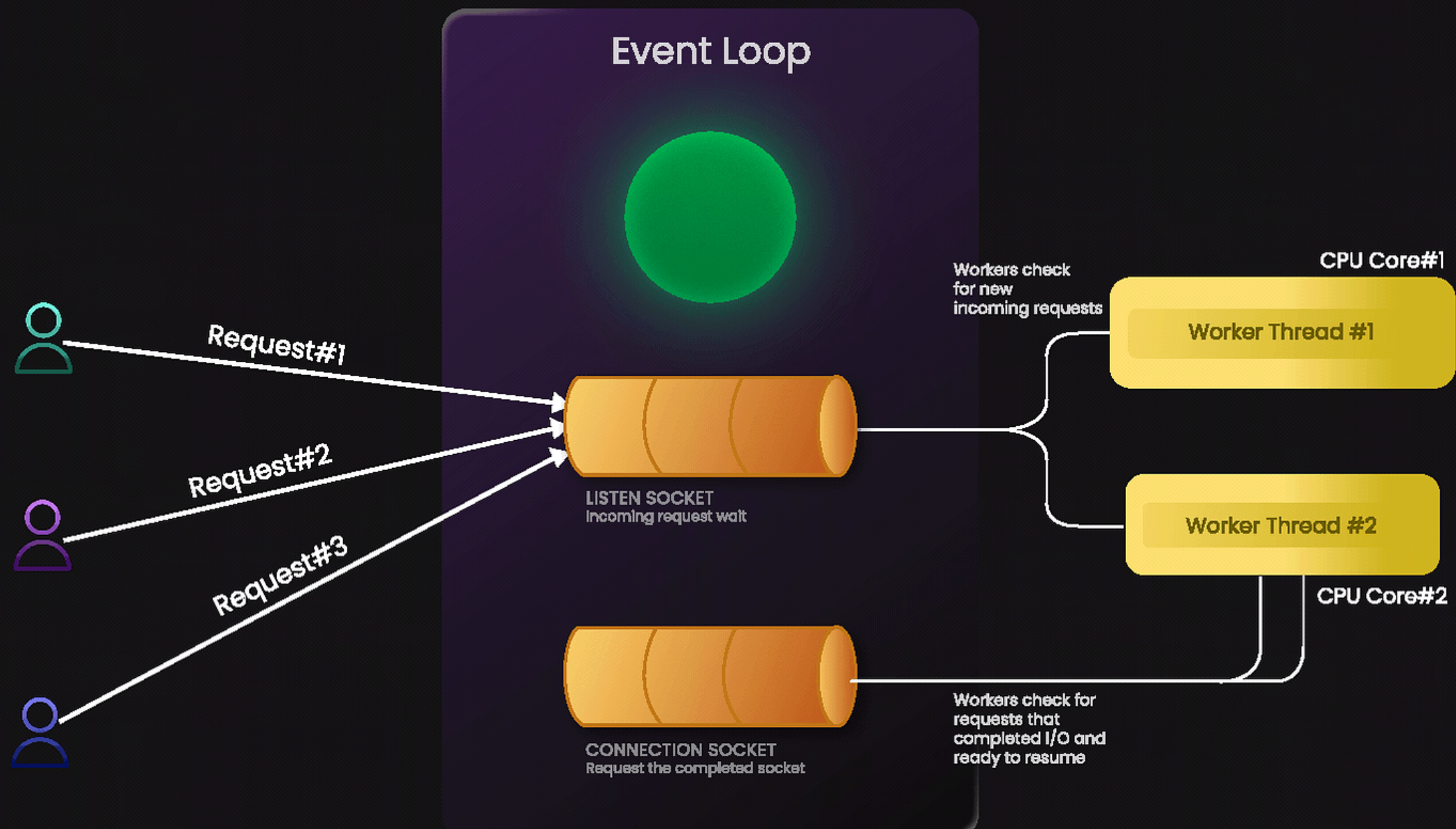
- FastAPI Pods: 6개
- Redis Pods: 3개
- Nginx ingress : 1개
- 총 CPU 요청: 8 CPU

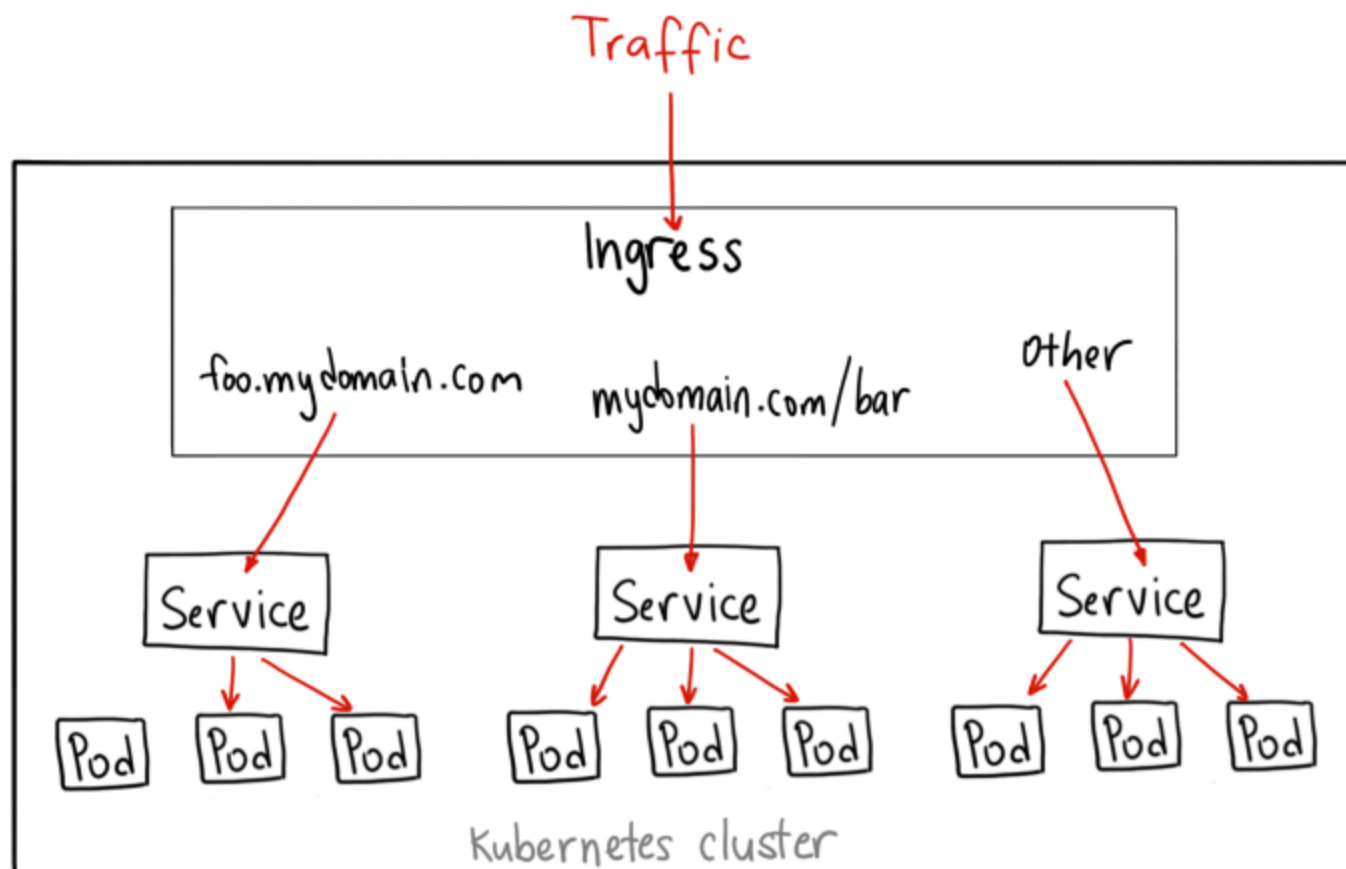
Kubernetes 배포

- HPA: CPU/Memory 기반 자동 확장
- PDB: 최소 가용 파드 보장
- Nginx Ingress: 고성능 로드밸런싱

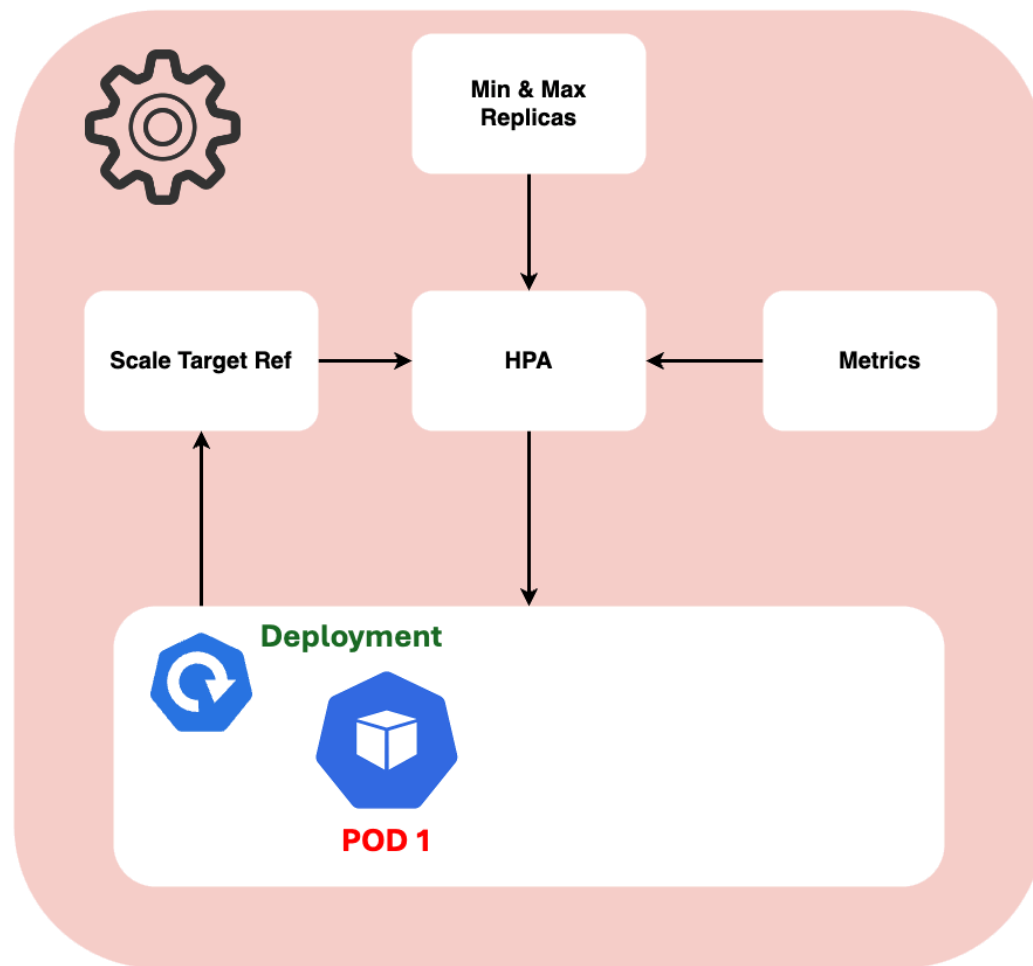
도커나 쿠버네티스에 대해 어느정도 아시나요?

- 엄밀한 정의는 아니지만 쉽게 도커는 가상 컴퓨터 하나
- 쿠버네티스는 여러 가상 컴퓨터들을 관리(자동 배포, 로드밸런싱, 스케일링)
- docker-compose랑은 가상 컴퓨터 클러스터와 개수를 유연하게 관리하고 싶어서 만들어진 도구

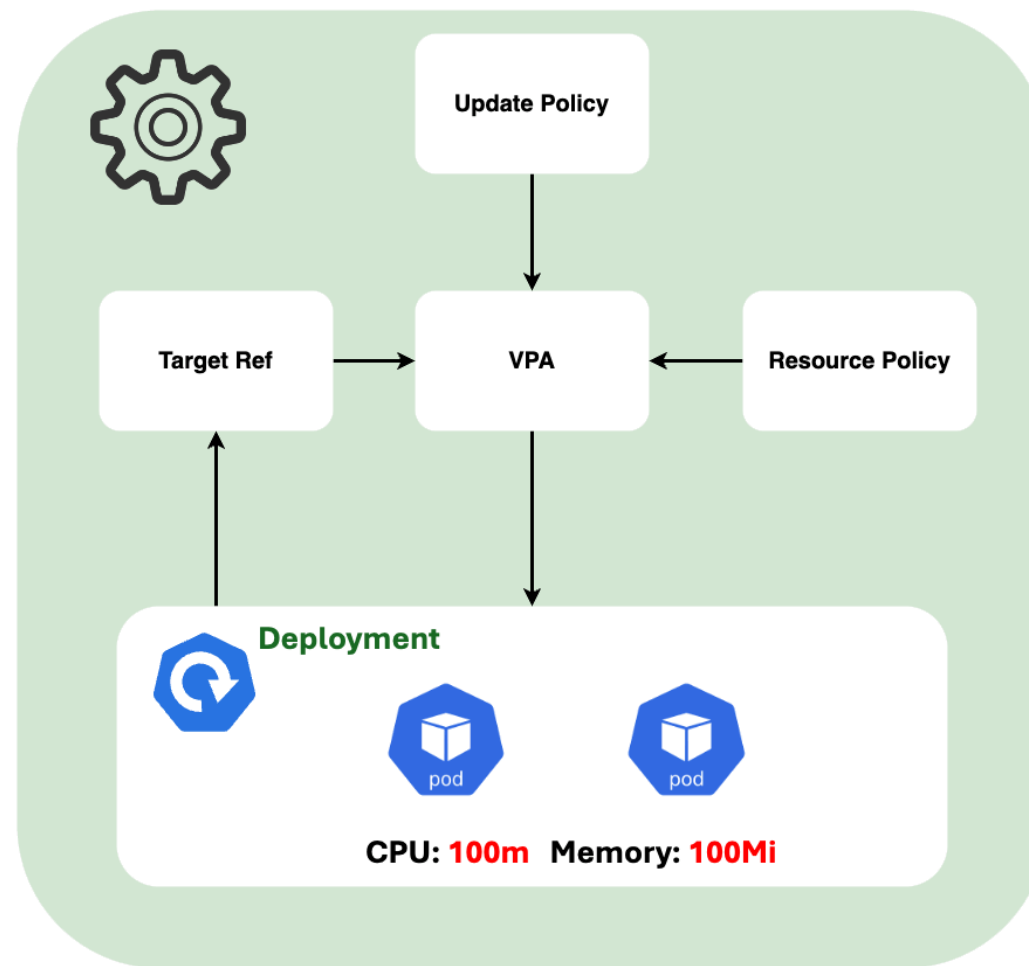




Horizontal Pod Autoscaling (HPA)



Vertical Pod Autoscaling (HPA)



부하 테스트 결과

Locust 테스트 설정

- 동시 사용자: 2000명
- 연결 풀: 12000개 동시 연결
- 타임아웃: 120초
- Keep-Alive 최적화

성능 결과

- 달성 RPS: 9000 (목표: 24000)
- 응답 시간: 평균 100ms 이내 유지

주요 트러블슈팅

네트워크 최적화

- Minikube SSH FD 고갈 해결
- Nginx Ingress 포트 포워딩 안정화

애플리케이션 최적화

- uvloop, httptools 적용
- Gunicorn 워커 수 튜닝
- Redis 연결 풀 최적화

인프라 안정성

- Health Check 강화
- 자동 확장 정책 수립

핵심 학습 포인트

자료구조 선택의 중요성

- 관계형 DB → 트라이: 성능 획기적 개선
- 시간복잡도: $O(n \log n)$ → $O(1)$

읽기/쓰기 분리

- 트라이 캐시(읽기) + 트라이 DB(쓰기)
- 높은 QPS 처리 가능

시스템 설계의 중요성

- 요구사항 분석을 엄밀히 하기
- 인프라 최적화의 중요성